Chess Engine with Hybrid AI Architecture - Project Proposal

Team Members: Shubham Kumar, Amritasingh Thakur

Project Overview

We propose to develop a chess engine that combines classical game-tree search algorithms with modern neural network-based board evaluation. Our goal is to create an AI system that demonstrates strong tactical play through search algorithms while learning strategic position evaluation from expert games.

Technical Approach

Phase 1: Classical Search Implementation We will implement a Minimax algorithm enhanced with Alpha-Beta pruning and iterative deepening to efficiently explore the game tree. Our baseline will search 5-6 moves ahead using hand-crafted heuristics for board evaluation (material count, piece positioning, king safety, etc.). Move ordering optimizations (checks > captures > threats) will improve pruning efficiency.

Phase 2: Neural Network Integration We will train a small recurrent neural network to replace the hand-crafted evaluation function. The model will take chess board states (converted to bitboard representations or tensors) as input and output a scalar evaluation score. Training data will consist of 10,000+ grandmaster games from the Lichess Elite Database (KingBase PGN format), with positions labeled by game outcomes (-1, 0, +1).

Phase 3: Hybrid System We will integrate the trained neural network as the evaluation function within our Alpha-Beta search framework. To manage computational costs, we will limit NN inference calls during deep searches, potentially using the classical heuristic for rapid preliminary evaluations and the NN for critical positions.

Tools and Technologies

- **Language:** Python
- **Chess Library:** python-chess (move generation, game state management)
- **Frontend:** Streamlit (interactive UI with win probability visualization)
- **ML Framework:** PyTorch (neural network implementation and training)
- **Dataset:** KingBase (Lichess Elite Database) PGN files

Expected Outcomes

By the end of the project, we aim to demonstrate that our hybrid chess engine significantly outperforms the purely heuristic-based version (Week 3 baseline vs. Week 8 final engine). Success will be measured through head-to-head matches and evaluation accuracy on test positions. We will deliver complete documentation, a playable web interface, and comparative analysis of different architectural approaches.

This project combines adversarial search (covered in the course) with practical machine learning, providing hands-on experience with both classical AI techniques and modern deep learning in a constrained, well-defined problem space.

Project Plan with Delivierables -

Month 1 (Feb)

week 1 - Environment & Baseline

→ Implement Random & Greedy mover
→ python-chess and streamlit
→ main.py set up input and output
play against random agent.

week 2 - Adverserial search

① Adversarial Search (Minimax + αβ pruning)
→ Look 3-4 moves ahead
→ Goal = Don't hang pieces instantly.

week 3 - Implement Heuristics

① Use heuristics to define good/bad positions
② write evaluate_board (board)
↳ pawn = 1, knight = 3, rook = 5
↳ Knights are better in center
↳ Rooks should be connected

week 4 - Complexity Optimization

→ speed up the search to reach L5 or 6.
→ Implement move ordering
↳ checks > captures > threats

Language → python

Library → python-chess

Frontend → Streamlit

Backend → αβ, Minimax
↳ Neural Network
↳ Training Dataset.

Chess Engine

Phase 1

Minimax Algorithm
Alpha Beta Pruning
Iterative Deepening

Phase 2

Neural Network
Training Model

Data Source

- Lichess Elite Database
- KingBase (PGN)

Month 2 (March)

week 5 - Data Pipeline & training

→ prepare data to train the brain.
→ 10k+ PGN games → parse using python-chess
→ Convert board states → bit boards / tensors
Output = (-1, 0, 1)
Input ←

week 6 - Neural Network (ML)

→ Replace evaluate_board() → Model above.
→ Small KNN that takes board state
And outputs a scalar score (e.g. +1.5)
→ train on my dataset

week 7 - Integration (Hybrid systems)

→ Goal ⇒ Combine Search + NN
→ Plug trained pyTorch model into αβ search
→ Note NN inference is slower than Math
↳ Limit NN calls

week 8 - Final Polish and Report

→ Add a win probability bar in Streamlit
→ Play this Engine Vs week 3 engine
→ ML engine should win.
→ complete all documentation